

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
```

```
from google.colab import drive
drive.mount('/content/drive')
from google.colab import files
```

Mounted at /content/drive

✧ Import data dari Drive

```
df = pd.read_csv('/content/drive/MyDrive/Dataset/social_media_screentime_mental_health_2026.csv')
```

df

[Show hidden output](#)

✧ Cek Struktur Data

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7000 entries, 0 to 6999
Data columns (total 25 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   participant_id                        7000 non-null   object
 1   age                                   7000 non-null   int64
 2   gender                               6930 non-null   object
 3   occupation                           7000 non-null   object
 4   region                               7000 non-null   object
 5   most_used_platform                   7000 non-null   object
 6   platforms_used_count                 7000 non-null   int64
 7   daily_screen_hours                   7000 non-null   float64
 8   daily_notifications                  7000 non-null   int64
 9   night_time_use                       7000 non-null   object
10   minutes_to_first_check_after_waking 7000 non-null   int64
11   primary_purpose                        7000 non-null   object
12   avg_sleep_hours                      6895 non-null   float64
13   anxiety_score_0to27                  7000 non-null   int64
14   low_mood_score_0to27                  7000 non-null   int64
15   life_satisfaction_1to10               7000 non-null   int64
16   loneliness_1to10                     7000 non-null   int64
17   self_esteem_1to10                    7000 non-null   int64
18   fomo_1to10                           7000 non-null   int64
19   social_comparison_1to10              7000 non-null   int64
20   physical_activity_days_per_week       7000 non-null   int64
21   uses_screen_time_limits              7000 non-null   object
22   attempted_digital_detox              7000 non-null   object
23   seeks_mental_health_support          7000 non-null   object
24   wellbeing_band                       7000 non-null   object
dtypes: float64(2), int64(12), object(11)
memory usage: 1.3+ MB
```

```
df.columns
```

```
Index(['participant_id', 'age', 'gender', 'occupation', 'region',
      'most_used_platform', 'platforms_used_count', 'daily_screen_hours',
      'daily_notifications', 'night_time_use',
      'minutes_to_first_check_after_waking', 'primary_purpose',
      'avg_sleep_hours', 'anxiety_score_0to27', 'low_mood_score_0to27',
      'life_satisfaction_1to10', 'loneliness_1to10', 'self_esteem_1to10',
      'fomo_1to10', 'social_comparison_1to10',
      'physical_activity_days_per_week', 'uses_screen_time_limits',
      'attempted_digital_detox', 'seeks_mental_health_support',
      'wellbeing_band'],
      dtype='object')
```

✓ Cek Quality Data

✓ Data Kosong

```
#cek data kosong
```

```
df.isnull().sum()
```

	0
participant_id	0
age	0
gender	70
occupation	0
region	0
most_used_platform	0
platforms_used_count	0
daily_screen_hours	0
daily_notifications	0
night_time_use	0
minutes_to_first_check_after_waking	0
primary_purpose	0
avg_sleep_hours	105
anxiety_score_0to27	0
low_mood_score_0to27	0
life_satisfaction_1to10	0
loneliness_1to10	0
self_esteem_1to10	0
fomo_1to10	0
social_comparison_1to10	0
physical_activity_days_per_week	0
uses_screen_time_limits	0
attempted_digital_detox	0
seeks_mental_health_support	0
wellbeing_band	0

```
dtype: int64
```

mengisi data kosong dengan modus

```
modus_gender = df['gender'].mode()[0]
df['gender'] = df['gender'].fillna(modus_gender)

modus_sleep_hours = df['avg_sleep_hours'].mode()[0]
df['avg_sleep_hours'] = df['avg_sleep_hours'].fillna(modus_sleep_hours)
```

✓ Data Duplikat

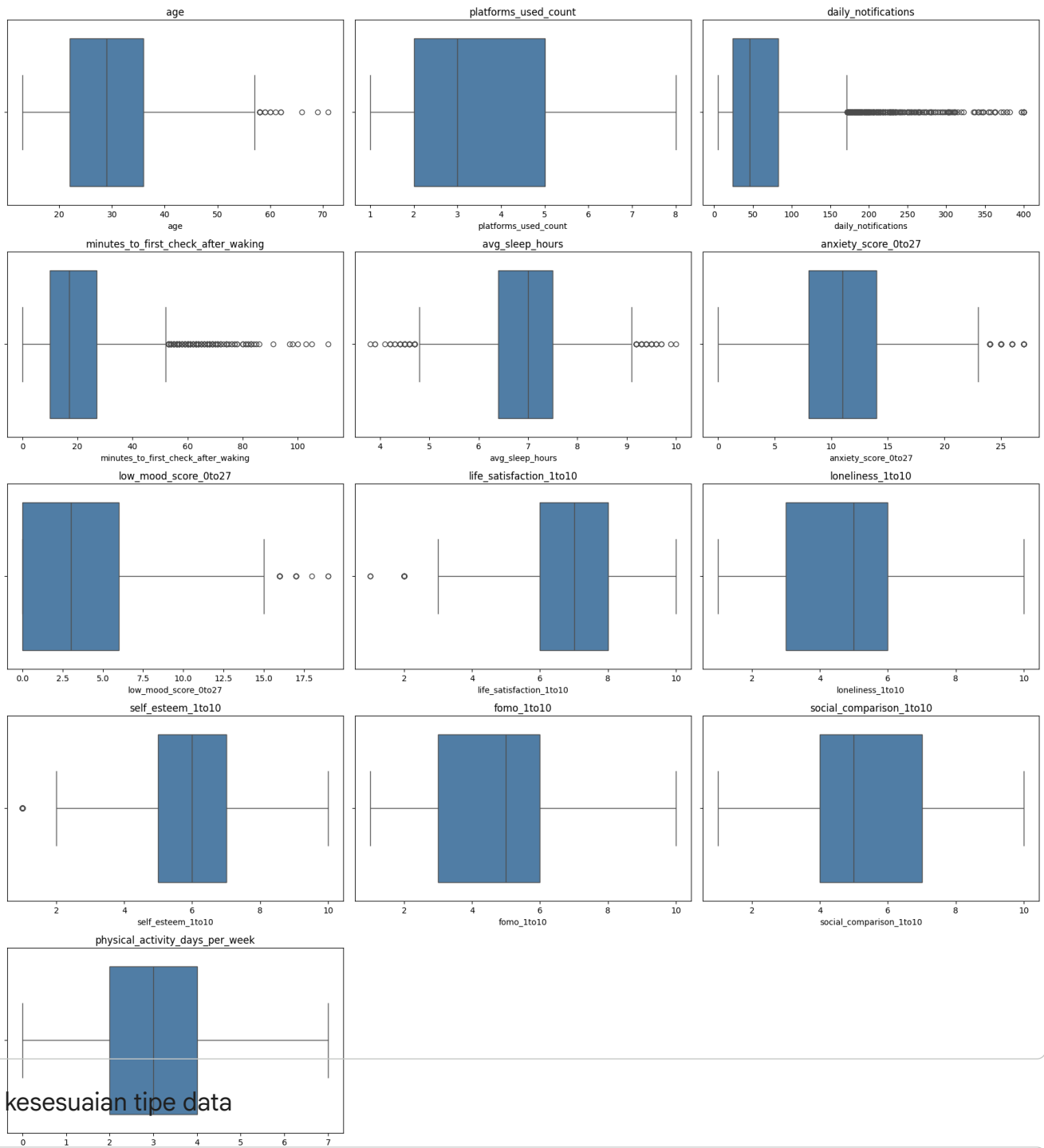
```
#cek data duplikat
```

```
df.duplicated().sum()
```

```
np.int64(0)
```

✓ Data Outlier

```
kolom_numerikal = [  
    'age',  
    'platforms_used_count',  
    'daily_notifications',  
    'minutes_to_first_check_after_waking',  
    'avg_sleep_hours',  
    'anxiety_score_0to27',  
    'low_mood_score_0to27',  
    'life_satisfaction_1to10',  
    'loneliness_1to10',  
    'self_esteem_1to10',  
    'fomo_1to10',  
    'social_comparison_1to10',  
    'physical_activity_days_per_week'  
]  
  
fig, axes = plt.subplots(nrows = 5, ncols=3, figsize = (18,20))  
axes = axes.flatten()  
  
for i, kolom in enumerate(kolom_numerikal):  
    sns.boxplot(data=df, x=kolom, ax=axes[i], color='steelblue')  
    axes[i].set_title(kolom)  
  
# Hide any unused subplots  
for j in range(len(kolom_numerikal), len(axes)):  
    axes[j].set_visible(False)  
  
plt.tight_layout()  
plt.savefig('boxplot.png', dpi=150, bbox_inches='tight')  
plt.show()  
  
files.download('boxplot.png')
```



✓ cek kesesuaian tipe data

df.dtypes

	0
participant_id	object
age	int64
gender	object
occupation	object
region	object
most_used_platform	object
platforms_used_count	int64
daily_screen_hours	float64
daily_notifications	int64
night_time_use	object
minutes_to_first_check_after_waking	int64
primary_purpose	object
avg_sleep_hours	float64
anxiety_score_0to27	int64
low_mood_score_0to27	int64
life_satisfaction_1to10	int64
loneliness_1to10	int64
self_esteem_1to10	int64
fomo_1to10	int64
social_comparison_1to10	int64
physical_activity_days_per_week	int64
uses_screen_time_limits	object
attempted_digital_detox	object
seeks_mental_health_support	object
wellbeing_band	object

dtype: object

Statistik Deskriptif

df.describe()

	age	platforms_used_count	daily_screen_hours	daily_notifications	minutes_to_first_check_after_waking	avg_sleep
count	7000.000000	7000.000000	7000.000000	7000.000000	7000.000000	7000
mean	29.246571	3.494571	3.304971	61.528286	20.008286	6
std	9.569132	1.763637	1.881350	54.879070	13.976229	0
min	13.000000	1.000000	0.200000	5.000000	0.000000	3
25%	22.000000	2.000000	1.900000	24.000000	10.000000	6
50%	29.000000	3.000000	3.000000	45.500000	17.000000	7
75%	36.000000	5.000000	4.300000	83.000000	27.000000	7
max	71.000000	8.000000	14.000000	400.000000	111.000000	10

df.describe(include='object')

	participant_id	gender	occupation	region	most_used_platform	night_time_use	primary_purpose	uses_screen_time_limits
count	7000	7000	7000	7000	7000	7000	7000	7000
unique	7000	4	6	6	8	4	6	2
top	P306999	Male	Student	Asia	TikTok	Sometimes	Entertainment	No
freq	1	3331	2715	1959	1733	2616	2147	4811

```
df['gender'].mode()[0]
```

```
'Male'
```

```
df['avg_sleep_hours'].mode()
```

```
avg_sleep_hours
0          7.0
```

```
dtype: float64
```

✖ Analisis Univariate

✖ Numerikal

kolom_numerikal:

- 1. age
- 2. platforms_used_count
- 3. daily_notifications
- 4. minutes_to_first_check_after_waking
- 5. avg_sleep_hours
- 6. anxiety_score_0to27
- 7. low_mood_score_0to27
- 8. life_satisfaction_1to10
- 9. loneliness_1to10
- 10. self_esteem_1to10
- 11. fomo_1to10
- 12. social_comparison_1to10
- 13. physical_activity_days_per_week

```
kolom_numerikal = [
    'age',
    'platforms_used_count',
    'daily_notifications',
    'minutes_to_first_check_after_waking',
    'avg_sleep_hours',
    'anxiety_score_0to27',
    'low_mood_score_0to27',
    'life_satisfaction_1to10',
    'loneliness_1to10',
    'self_esteem_1to10',
    'fomo_1to10',
    'social_comparison_1to10',
    'physical_activity_days_per_week'
]

fig,axes = plt.subplots(nrows = 5, ncols=3, figsize = (18,20))
axes = axes.flatten()

for i, kolom in enumerate(kolom_numerikal):
    sns.histplot(data=df, x=kolom, kde=False, ax=axes[i], color='steelblue', edgecolor='black')
```

```

    axes[i].set_title(kolom)

# Hide any unused subplots
for j in range(len(kolom_numerikal), len(axes)):
    axes[j].set_visible(False)

plt.tight_layout()
plt.savefig('histogram.png', dpi=150, bbox_inches='tight')
plt.show()

files.download('histogram.png')

```

[Show hidden output](#)

✶ Kategorikal

kolom_kategorikal:

1. participant_id
2. gender
3. occupation
4. region
5. most_used_platform
6. night_time_use
7. primary_purpose
8. uses_screen_time_limits
9. attempted_digital_detox
10. seeks_mental_health_support
11. wellbeing_band

```

kolom_kategorikal = [
    'gender',
    'occupation',
    'region',
    'most_used_platform',
    'night_time_use',
    'primary_purpose',
    'uses_screen_time_limits',
    'attempted_digital_detox',
    'seeks_mental_health_support',
    'wellbeing_band'
]

fig, axes = plt.subplots(nrows=4, ncols=3, figsize=(18, 20))
axes = axes.flatten() # biar bisa diakses pakai index [0], [1], dst

for i, kolom in enumerate(kolom_kategorikal):
    df[kolom].value_counts().plot(kind='bar', ax=axes[i], color='steelblue', edgecolor='black')
    axes[i].set_title(kolom)
    axes[i].set_xlabel('')
    axes[i].tick_params(axis='x', rotation=45)

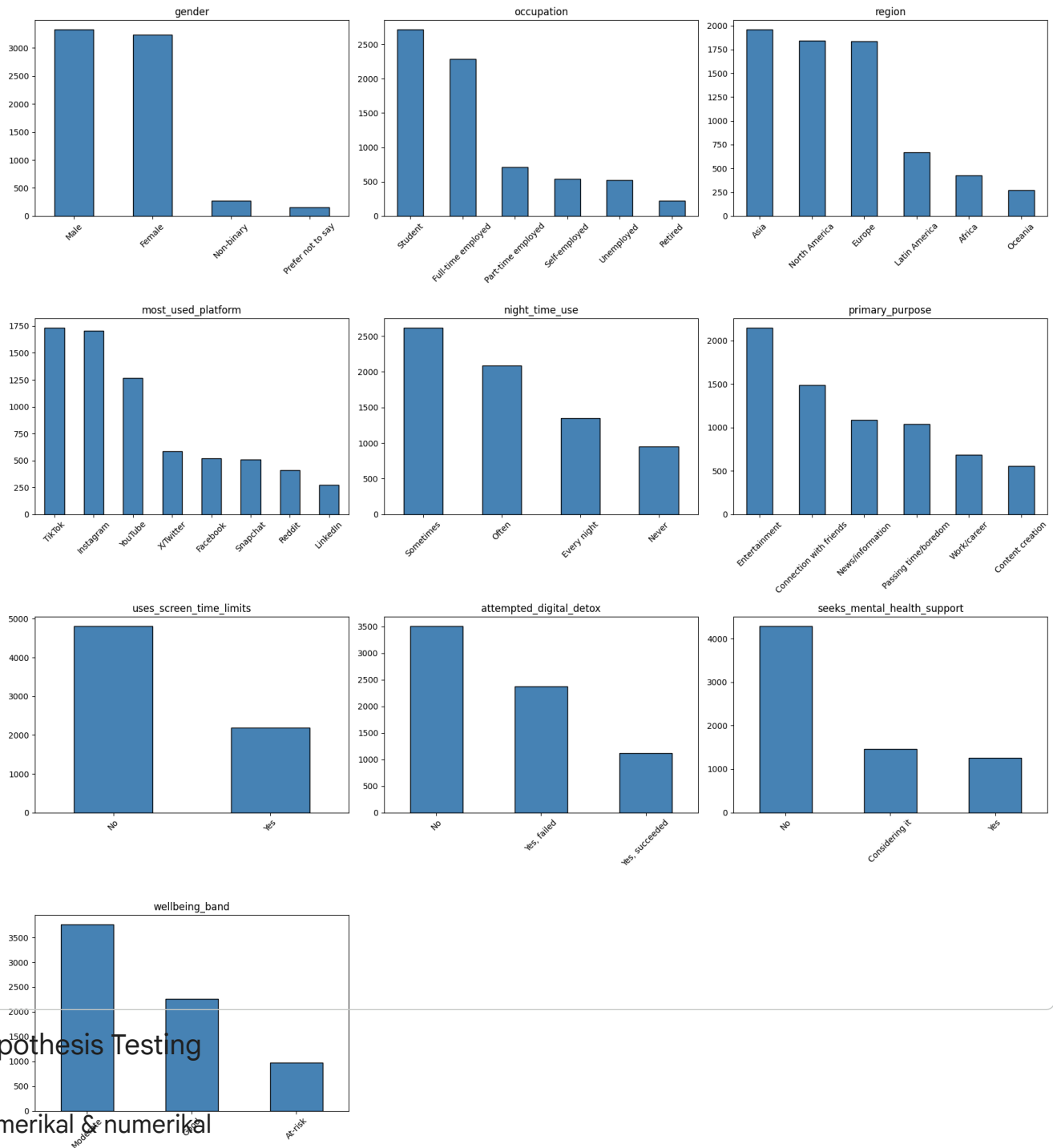
# Hide any unused subplots
for j in range(len(kolom_kategorikal), len(axes)):
    axes[j].set_visible(False)

plt.suptitle('Distribusi Kolom Kategorikal', fontsize=16, y=1.02)
plt.tight_layout()
plt.savefig('bar_charts.png', dpi=150, bbox_inches='tight')
plt.show()

files.download('bar_charts.png')

```

Distribusi Kolom Kategorikal



✓ Hypothesis Testing

✓ Numerikal & numerikal

Apakah Usia ada hubungannya sama aktivitas harian selama seminggu?

H0 = tidak ada hubungan antara age dengan aktivitas harian selama seminggu (korelasi = 0) H1 = ada hubungan antara age dengan aktivitas harian selama seminggu (korelasi = 1)


```
# melakukan tes distribusi

from scipy import stats

# Cek normalitas
stat_age, p_age = stats.shapiro(df['age'])
stat_activity, p_activity = stats.shapiro(df['physical_activity_days_per_week'])

print(f"Age normal? p-value: {p_age:.4f}")
print(f"Activity normal? p-value: {p_activity:.4f}")
```

```
Age normal? p-value: 0.0000
Activity normal? p-value: 0.0000
/usr/local/lib/python3.12/dist-packages/scipy/stats/_axis_nan_policy.py:579: UserWarning: scipy.stats.shapiro: For N > 5000, com
res = hypotest_fun_out(*samples, **kwargs)
```

```
corr, p_value = stats.spearmanr(df['age'], df['physical_activity_days_per_week'])

print(f"Korelasi Spearman: {corr:.4f}")
print(f"p-value: {p_value:.4f}")

if p_value < 0.05:
    print("Tolak H0 - ada hubungan signifikan antara usia dan aktivitas fisik")
else:
    print("Gagal tolak H0 - tidak cukup bukti ada hubungan")
```

```
Korelasi Spearman: -0.0184
p-value: 0.1232
Gagal tolak H0 - tidak cukup bukti ada hubungan
```

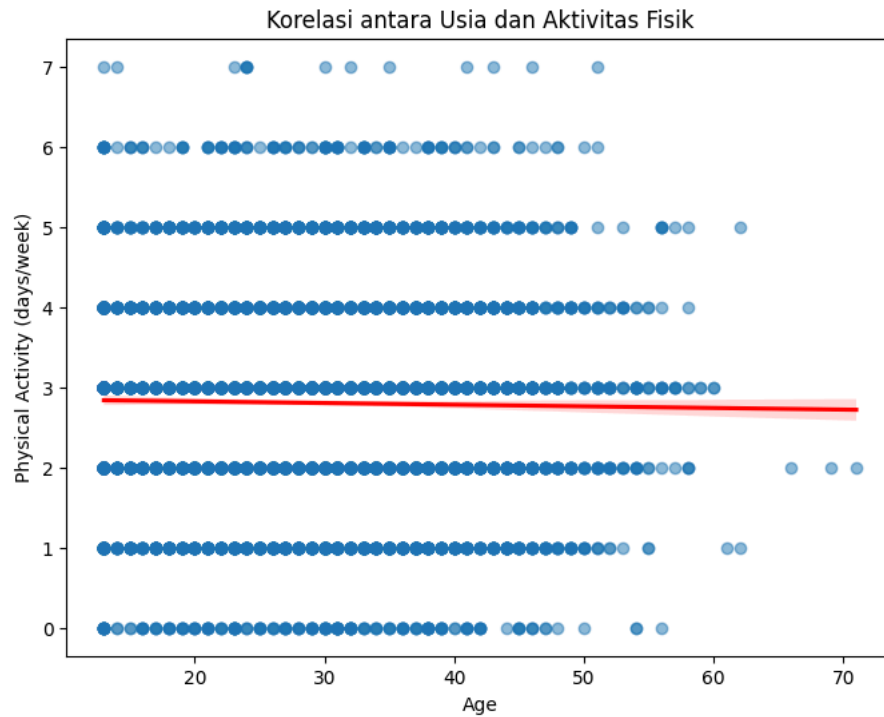
```
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

# Scatter plot dengan regression line
plt.figure(figsize=(8,6))
sns.regplot(data=df, x='age', y='physical_activity_days_per_week',
            scatter_kws={'alpha':0.5}, line_kws={'color':'red'})

plt.xlabel('Age')
plt.ylabel('Physical Activity (days/week)')
plt.title('Korelasi antara Usia dan Aktivitas Fisik')
plt.savefig('hypothesis 1.png', dpi=150, bbox_inches='tight')
plt.show()

files.download('hypothesis 1.png')

# Hitung korelasi Spearman (karena data kamu nggak normal)
corr, p_value = stats.spearmanr(df['age'], df['physical_activity_days_per_week'])
print(f"Korelasi Spearman: {corr:.4f}")
print(f"p-value: {p_value:.4f}")
```



✓ Numerikal & kategorikal

Uji tes anova antara gender dengan anxiety score

```
from scipy import stats

groups = [df[df['gender'] == g]['anxiety_score_0to27'] for g in df['gender'].unique()]

f_stat, p_value = stats.f_oneway(*groups)

print(f"F-statistic : {f_stat:.4f}")
print(f"P-value      : {p_value:.4f}")

if p_value < 0.05:
    print("✅ Ada perbedaan signifikan anxiety score antar gender")
else:
    print("❌ Tidak ada perbedaan signifikan")
```

```
F-statistic : 0.4616
P-value      : 0.7091
❌ Tidak ada perbedaan signifikan
```

```
# Boxplot
sns.boxplot(data=df, x='gender', y='anxiety_score_0to27', palette='Set2')
plt.title('Anxiety Score berdasarkan Gender')
plt.savefig('hypothesis 2,1.png', dpi=150, bbox_inches='tight')
plt.show()

files.download('hypothesis 2,1.png')

# Rata-rata per gender
df.groupby('gender')['anxiety_score_0to27'].mean().plot(kind='bar', color='steelblue', edgecolor='black')
plt.title('Rata-rata Anxiety Score per Gender')
plt.ylabel('Anxiety Score')
plt.xticks(rotation=45)

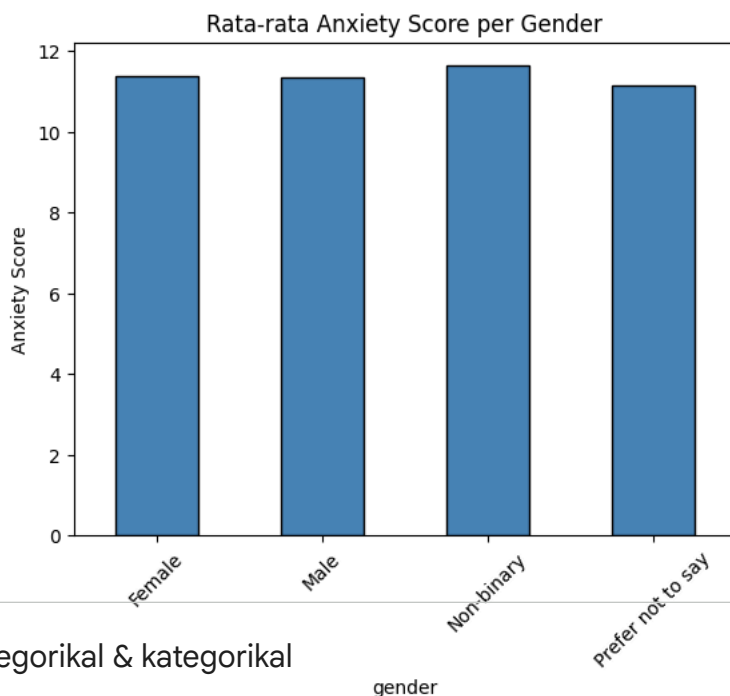
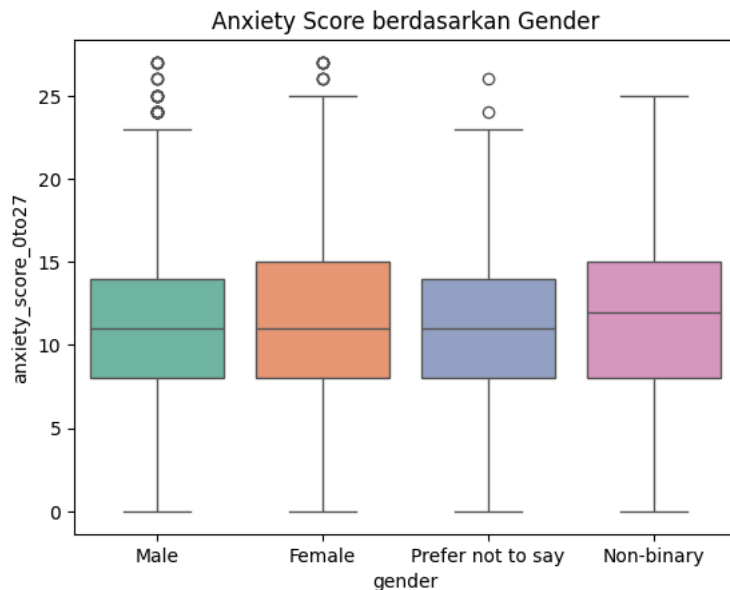
plt.savefig('hypothesis 2,2.png', dpi=150, bbox_inches='tight')
plt.show()
```

```
files.download('hypothesis 2,2.png')
```

```
/tmp/ipykernel_777/762671022.py:2: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set
```

```
sns.boxplot(data=df, x='gender', y='anxiety_score_0to27', palette='Set2')
```



✦ Kategorikal & kategorikal

Melakukan hypothesis testing apakah ada hubungan antara region dengan penggunaan limit screen time?

```
# 1 cek isi kedua kolom
```

```
print(df['region'].value_counts())
print(df['uses_screen_time_limits'].value_counts())
```

```
region
Asia      1959
North America  1842
Europe    1834
Latin America  670
Africa     425
Oceania    270
Name: count, dtype: int64
uses_screen_time_limits
No      4811
```

```
Yes    2189
Name: count, dtype: int64
```

```
# buat crosstab
```

```
crosstab = pd.crosstab(df['region'], df['uses_screen_time_limits'])
print(crosstab)
```

uses_screen_time_limits	No	Yes
region		
Africa	291	134
Asia	1338	621
Europe	1266	568
Latin America	475	195
North America	1252	590
Oceania	189	81

```
# Chi Square Test
```

```
from scipy.stats import chi2_contingency
```

```
chi2, p_value, dof, expected = chi2_contingency(crosstab)
```

```
print(f"Chi2 statistic : {chi2:.4f}")
print(f"P-value       : {p_value:.4f}")
print(f"Degrees of freedom: {dof}")
```

```
if p_value < 0.05:
```

```
    print("✅ Ada hubungan signifikan antara region dan uses_screen_time_limits")
```

```
else:
```

```
    print("❌ Tidak ada hubungan signifikan")
```

```
Chi2 statistic : 2.4183
```

```
P-value       : 0.7887
```

```
Degrees of freedom: 5
```

```
❌ Tidak ada hubungan signifikan
```

```
import numpy as np
```

```
n = crosstab.sum().sum() # total data
min_dim = min(crosstab.shape) - 1
cramers_v = np.sqrt(chi2 / (n * min_dim))
```

```
print(f"Cramer's V: {cramers_v:.4f}")
```

```
Cramer's V: 0.0186
```

```
import matplotlib.pyplot as plt
```

```
import pandas as pd
```

```
# Bikin contingency table dalam bentuk persentase
```

```
ct = pd.crosstab(df['region'], df['uses_screen_time_limits'], normalize='index') * 100
```

```
ct.plot(kind='bar', stacked=True, figsize=(10,6), color=['#ff9999', '#66b3ff'])
```

```
plt.xlabel('Region')
```

```
plt.ylabel('Persentase (%)')
```

```
plt.title('Proporsi Penggunaan Screen Time Limits per Region')
```

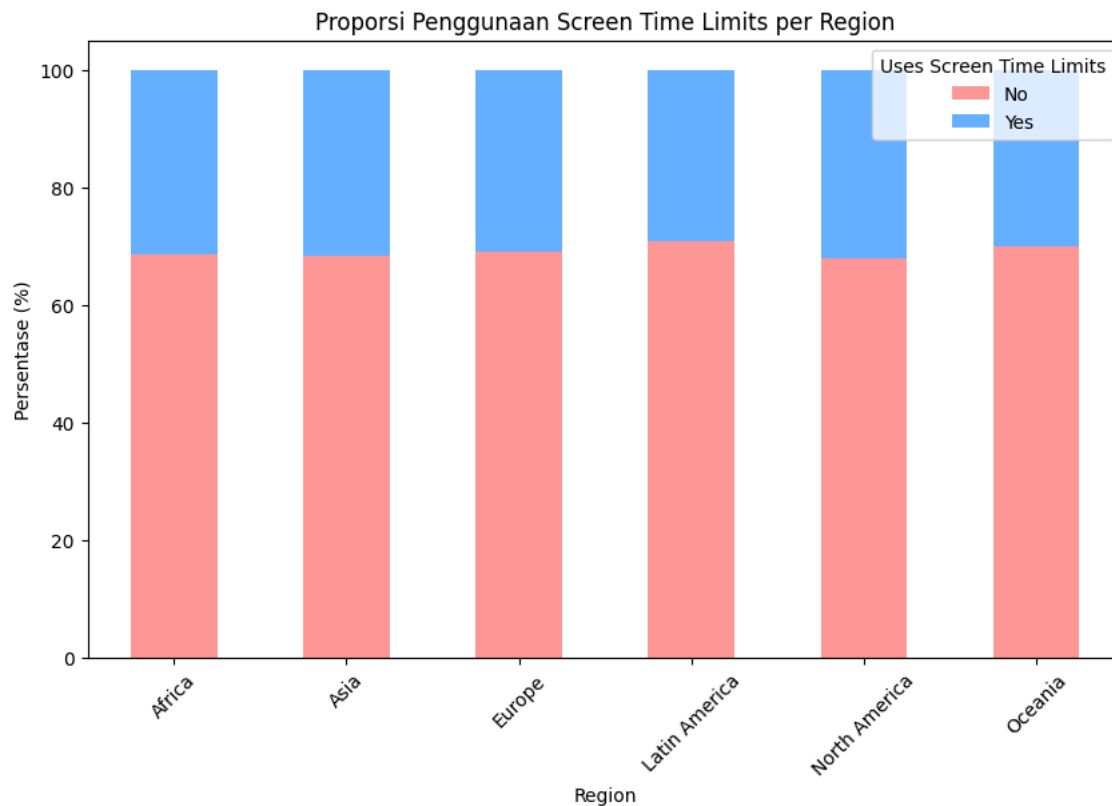
```
plt.legend(title='Uses Screen Time Limits')
```

```
plt.xticks(rotation=45)
```

```
plt.savefig('hypothesis 3.png', dpi=150, bbox_inches='tight')
```

```
plt.show()
```

```
files.download('hypothesis 3.png')
```



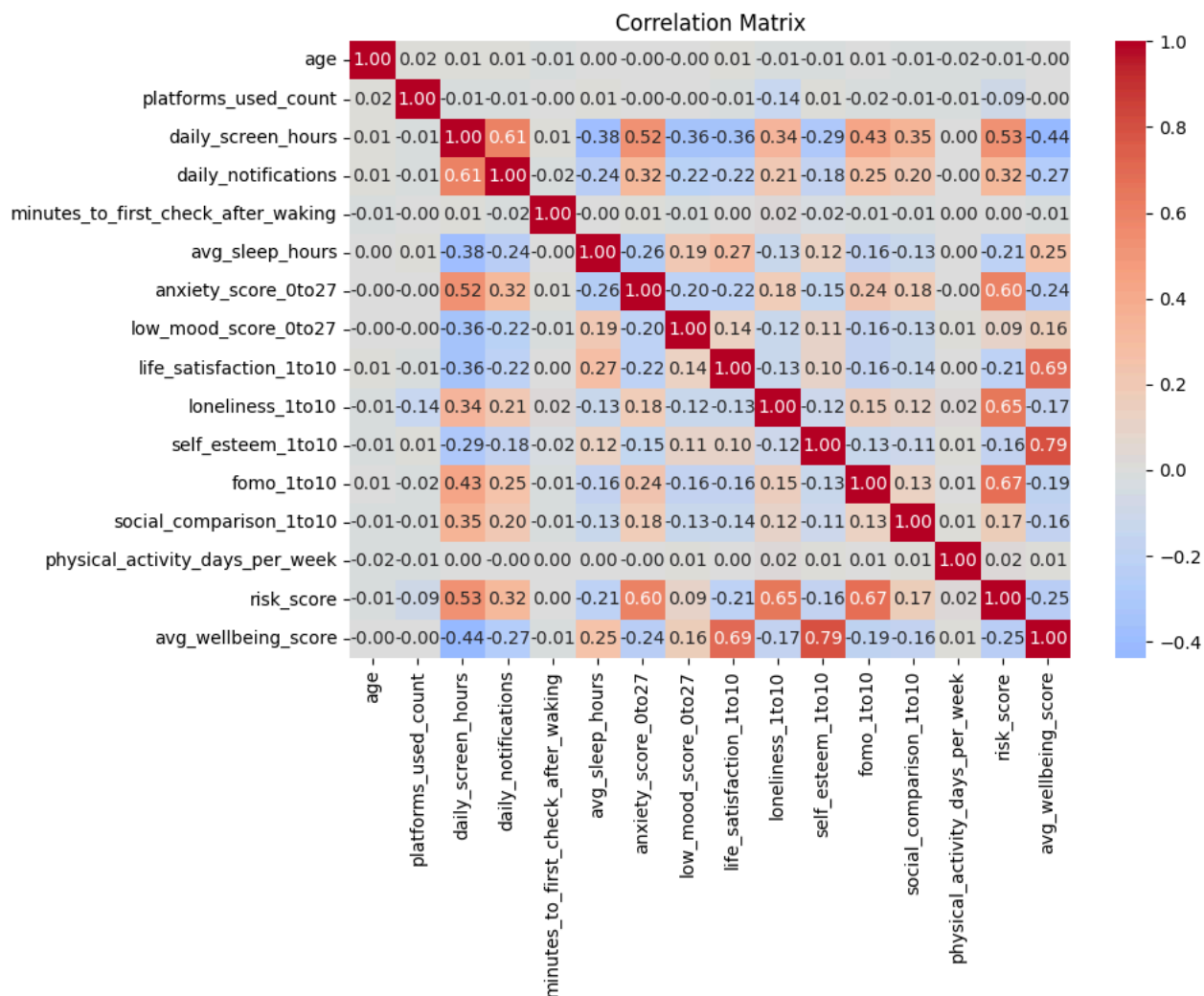
✓ Analisis Multivariate

```
# Pilih kolom numerik saja
kolom_numerik = df.select_dtypes(include='number')

# Hitung korelasi
corr_matrix = kolom_numerik.corr()

# Visualisasi heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(corr_matrix,
            annot=True,          # tampilkan angka korelasinya
            fmt='.2f',           # 2 desimal
            cmap='coolwarm',     # warna merah=positif, biru=negatif
            center=0)
plt.title('Correlation Matrix')
plt.tight_layout()
plt.savefig('heatmap correlation.png', dpi=150, bbox_inches='tight')
plt.show()

files.download('heatmap correlation.png')
```



```

from sklearn.preprocessing import LabelEncoder

df_encoded = df[kolom_kategorikal].copy()

le = LabelEncoder()
for col in kolom_kategorikal:
    df_encoded[col] = le.fit_transform(df_encoded[col].astype(str))

plt.figure(figsize=(12, 10))
sns.heatmap(df_encoded.corr(),
            annot=True,
            fmt='.2f',
            cmap='coolwarm',
            center=0)
plt.title('Correlation Heatmap - Kolom Kategorikal (Label Encoded)')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()

```

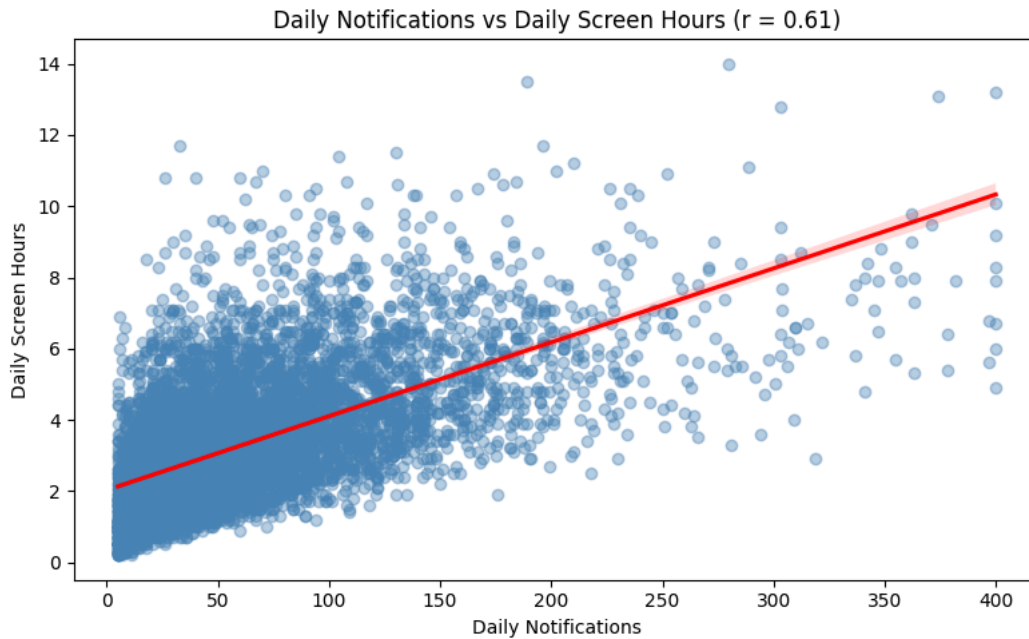


Analysis Bivariate

```
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 5))
sns.regplot(data=df,
            x='daily_notifications',
            y='daily_screen_hours',
            scatter_kws={'alpha': 0.4, 'color': 'steelblue'},
            line_kws={'color': 'red'})
plt.title('Daily Notifications vs Daily Screen Hours (r = 0.61)')
plt.xlabel('Daily Notifications')
plt.ylabel('Daily Screen Hours')
plt.tight_layout()
plt.savefig('scatter_plot.png', dpi=150, bbox_inches='tight')
plt.show()

files.download('scatter_plot.png')
```



```
from scipy import stats

# Cek normalitas dulu
stat1, p_norm1 = stats.shapiro(df['daily_notifications'].sample(500, random_state=42))
stat2, p_norm2 = stats.shapiro(df['daily_screen_hours'].sample(500, random_state=42))

print(f"Normalitas daily_notifications : p = {p_norm1:.4f}")
print(f"Normalitas daily_screen_hours : p = {p_norm2:.4f}")

Normalitas daily_notifications : p = 0.0000
Normalitas daily_screen_hours : p = 0.0000
```

Pada uji normalitas: H_0 = data berdistribusi normal H_1 = data tidak terdistribusi normal

Jadi aturan mainnya:

$p\text{-value} < 0.05$ (termasuk mendekati 0) $\rightarrow$ tolak $H_0 \rightarrow$ artinya data TIDAK normal

$p\text{-value} \geq 0.05 \rightarrow$ gagal tolak $H_0 \rightarrow$ data (kemungkinan) normal

Jadi kalau $p\text{-value}$ kamu 0, itu artinya datanya jauh dari normal, bukan normal.

```
# Kalau salah satu tidak normal  $\rightarrow$  Spearman
r, p_value = stats.spearmanr(df['daily_notifications'], df['daily_screen_hours'])

print(f"Correlation : {r:.4f}")
print(f"P-value : {p_value:.4f}")

if p_value < 0.05:
    print("✅ Korelasi signifikan secara statistik")
else:
    print("❌ Korelasi tidak signifikan")

Correlation : 0.6476
P-value : 0.0000
✅ Korelasi signifikan secara statistik
```

✧ Persiapan Pembuatan Dashboard

✧ KPI

Daily Screen Hours


```
rata_rata = df['daily_screen_hours'].mean()
print(rata_rata)
```

```
3.304971428571429
```

```
# Normalisasi dulu ke skala 0-1 sebelum digabung
df['risk_score'] = (
    df['anxiety_score_0to27']/27*10 +
    df['low_mood_score_0to27']/27*10 +
    df['loneliness_1to10'] +
    df['fomo_1to10']
) / 4

#df['risk_score'] = (df['risk_score'] - df['risk_score'].min()) / (df['risk_score'].max() - df['risk_score'].min())

df['risk_score']

average_risk_score = df['risk_score'].mean()
print(average_risk_score)
```

```
3.7398888888888884
```

```
df['avg_wellbeing_score'] = (
    df['life_satisfaction_1to10'] + df['self_esteem_1to10']
) / 2

average_wellbeing_score = df['avg_wellbeing_score'].mean()
print(average_wellbeing_score)
```

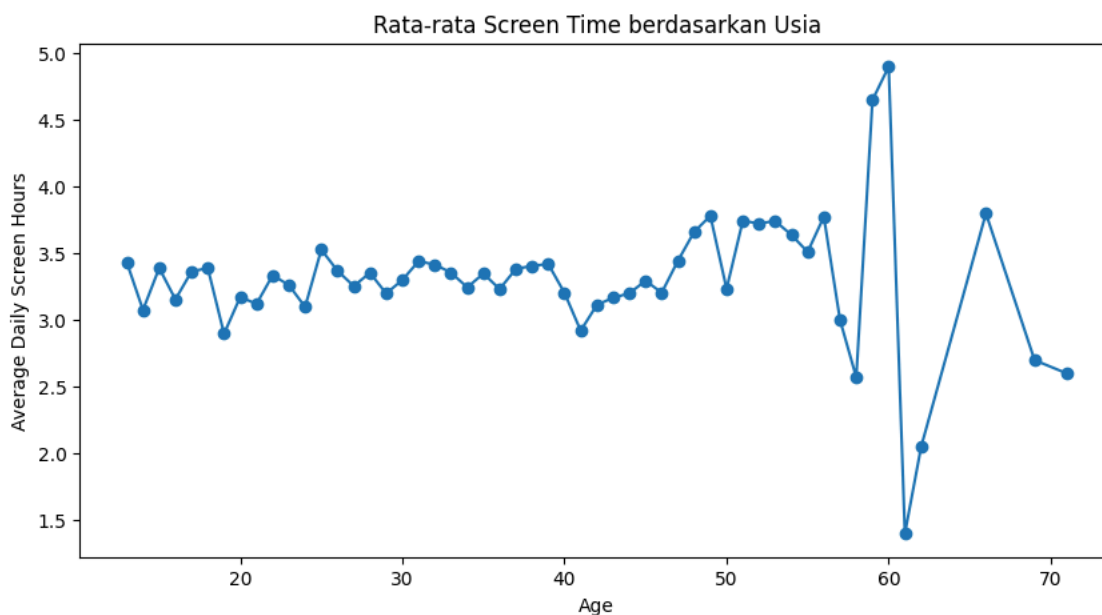
```
6.481357142857143
```

Line Chart

```
import matplotlib.pyplot as plt

age_screen = df.groupby('age')['daily_screen_hours'].mean()

plt.figure(figsize=(10,5))
plt.plot(age_screen.index, age_screen.values, marker='o')
plt.xlabel('Age')
plt.ylabel('Average Daily Screen Hours')
plt.title('Rata-rata Screen Time berdasarkan Usia')
plt.show()
```

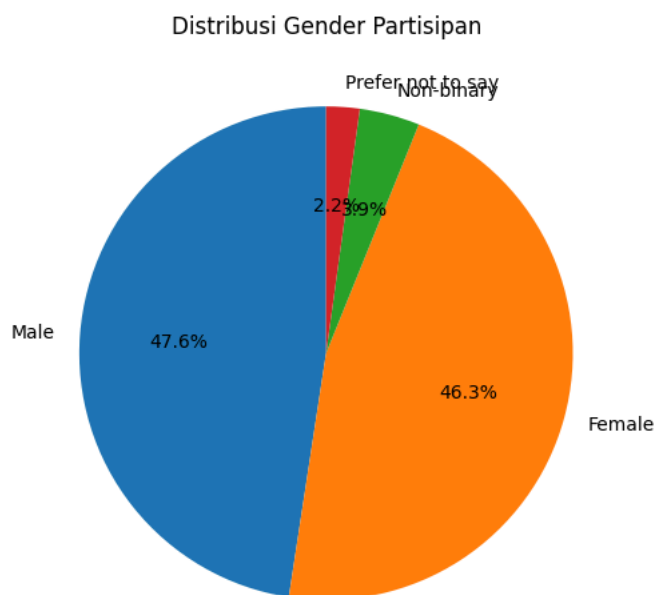


Pie Chart

```
import matplotlib.pyplot as plt

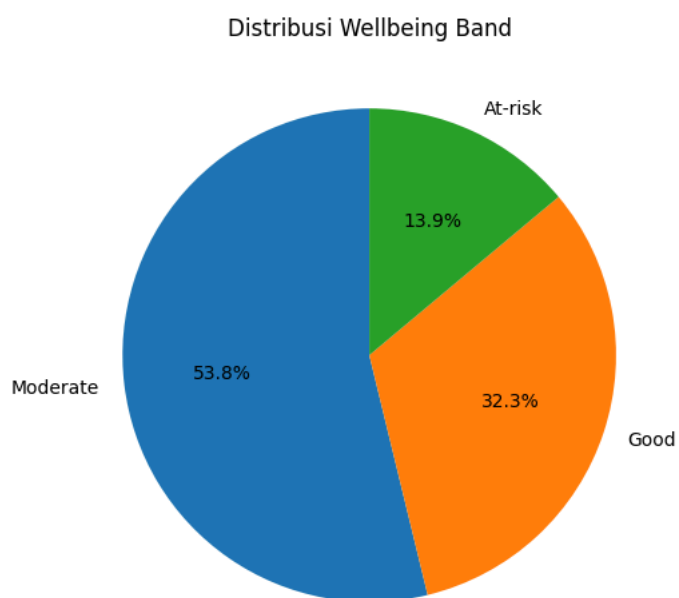
gender_counts = df['gender'].value_counts()

plt.figure(figsize=(6,6))
plt.pie(gender_counts, labels=gender_counts.index, autopct='%1.1f%%', startangle=90)
plt.title('Distribusi Gender Partisipan')
plt.show()
```



```
wellbeing_counts = df['wellbeing_band'].value_counts()

plt.figure(figsize=(6,6))
plt.pie(wellbeing_counts, labels=wellbeing_counts.index, autopct='%1.1f%%', startangle=90)
plt.title('Distribusi Wellbeing Band')
plt.show()
```

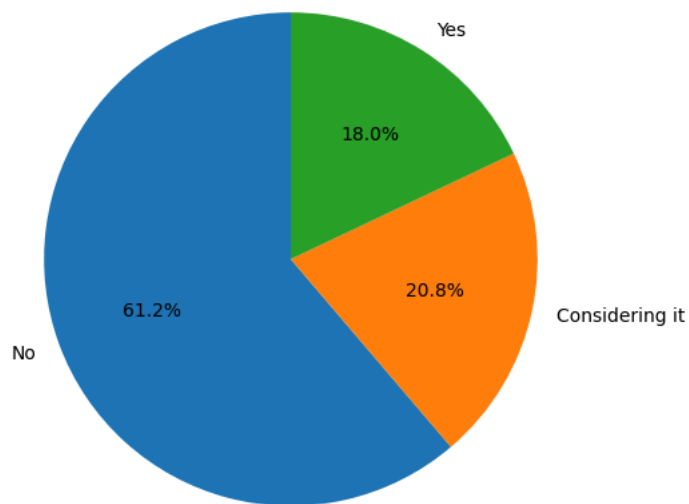


```
support_counts = df['seeks_mental_health_support'].value_counts()

plt.figure(figsize=(6,6))
plt.pie(support_counts, labels=support_counts.index, autopct='%1.1f%%', startangle=90)
```

```
plt.title('Partisipan yang Mencari Bantuan Mental Health')
plt.show()
```

Partisipan yang Mencari Bantuan Mental Health



Bar Chart

```
import matplotlib.pyplot as plt

platform_counts = df['most_used_platform'].value_counts()

plt.figure(figsize=(10,5))
platform_counts.plot(kind='bar', color='skyblue')
plt.xlabel('Platform')
plt.ylabel('Jumlah Partisipan')
plt.title('Platform Media Sosial yang Paling Sering Digunakan')
plt.xticks(rotation=45)
plt.show()
```

Platform Media Sosial yang Paling Sering Digunakan

